

File Edit View Run Kernel Tabs Settings Help

03_asl_cnn.ipynb

Python 3 (ipykernel)

Launcher

03_asl_cnn.ipynb

Code

Filter files by name

Name

Modified

data6s ago

images9s ago

03_asl_cnn.ipynbnext yr.

utils.pylast yr.



DEEP
LEARNING
INSTITUTE

3. Convolutional Neural Networks

In the previous section, we built and trained a simple model to classify ASL images. The model was able to learn how to correctly classify the training dataset with very high accuracy, but, it did not perform nearly as well on validation dataset. This behavior of not generalizing well to non-training data is called [overfitting](#), and in this section, we will introduce a popular kind of model called a [convolutional neural network](#) that is especially good for reading images and classifying them.

3.1 Objectives

- Prep data specifically for a CNN
- Create a more sophisticated CNN model, understanding a greater variety of model layers
- Train a CNN model and observe its performance

```
[1]: import torch.nn as nn
import pandas as pd
import torch
from torch.optim import Adam
from torch.utils.data import Dataset, DataLoader

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
torch.cuda.is_available()
```

[1]: True

3.2 Loading and Preparing the Data

3.2.1 Preparing Images

Let's begin by loading our DataFrames like we did in the previous lab:

```
[2]: train_df = pd.read_csv("data/asl_data/sign_mnist_train.csv")
valid_df = pd.read_csv("data/asl_data/sign_mnist_valid.csv")
```

This ASL data is already flattened.

```
[3]: sample_df = train_df.head().copy() # Grab the top 5 rows
sample_df.pop('label')
sample_x = sample_df.values
sample_x
```

```
[3]: array([[107, 118, 127, ..., 204, 203, 202],
[155, 157, 156, ..., 103, 135, 149],
[187, 188, 188, ..., 195, 194, 195],
[211, 211, 212, ..., 222, 229, 163],
[164, 167, 170, ..., 163, 164, 179]])
```

```
[4]: sample_x.shape
```

[4]: (5, 784)

In this format, we don't have all the information about which pixels are near each other. Because of this, we can't apply convolutions that will detect features. Let's [reshape](#) our dataset so that they are in a 28x28 pixel format. This will allow our convolutions to associate groups of pixels and detect important features.

Note that for the first convolutional layer of our model, we need to have not only the height and width of the image, but also the number of [color channels](#). Our images are grayscale, so we'll just have 1 channel.

That means that we need to convert the current shape `(5, 784)` to `(5, 1, 28, 28)`. With [NumPy](#) arrays, we can pass a `-1` for any dimension we wish to remain the same.

```
[5]: IMG_HEIGHT = 28
IMG_WIDTH = 28
IMG_CHS = 1

sample_x = sample_x.reshape(-1, IMG_CHS, IMG_HEIGHT, IMG_WIDTH)
sample_x.shape
```

[5]: (5, 1, 28, 28)

3.2.2 Create a Dataset

Let's add the steps above into our `MyDataset` class.

Exercise

There are 4 `FIXME`s in the class definition below. Can you replace them with the correct values?

```
[6]: class MyDataset(Dataset):
    def __init__(self, base_df):
        x_df = base_df.copy() # Some operations below are in-place
        y_df = x_df.pop(FIXME)
        x_df = x_df.values / 255 # Normalize values from 0 to 1
        x_df = x_df.reshape(-1, FIXME, FIXME, FIXME)
        self.xs = torch.tensor(x_df).float().to(device)
        self.ys = torch.tensor(y_df).to(device)
```

```
def __getitem__(self, idx):
    x = self.xs[idx]
    y = self.ys[idx]
    return x, y

def __len__(self):
    return len(self.xs)
```

Solution

Click the ... below for the solution.

SOLUTION ***

3.2.3 Create a DataLoader

Next, let's create the DataLoader from the Dataset

Exercise

One of these function calls is missing the `shuffle=True` argument. Can you remember which one it is and add it back in?

```
[8]: BATCH_SIZE = 32

train_data = MyDataset(train_df)
train_loader = DataLoader(train_data, batch_size=BATCH_SIZE)
train_N = len(train_loader.dataset)

valid_data = MyDataset(valid_df)
valid_loader = DataLoader(valid_data, batch_size=BATCH_SIZE)
valid_N = len(valid_loader.dataset)
```

Solution

Click the ... below for the solution.

SOLUTION ***

Let's grab a batch from the DataLoader to make sure it works.

```
[10]: batch = next(iter(train_loader))
      batch

[10]: [tensor([[[[0.4667, 0.4784, 0.4980, ..., 0.5882, 0.5882, 0.5804],
[0.4784, 0.4863, 0.5098, ..., 0.5882, 0.5882, 0.5804],
[0.4824, 0.4863, 0.5137, ..., 0.6039, 0.5961, 0.5843],
...,
[0.5725, 0.6039, 0.6314, ..., 0.7412, 0.7333, 0.7294],
[0.5647, 0.5961, 0.6275, ..., 0.7373, 0.7333, 0.7255],
[0.5647, 0.5961, 0.6275, ..., 0.7333, 0.7333, 0.7294]]],

[[[0.6588, 0.6627, 0.6667, ..., 0.6431, 0.6353, 0.6275],
[0.6627, 0.6627, 0.6706, ..., 0.6471, 0.6392, 0.6314],
[0.6667, 0.6706, 0.6745, ..., 0.6471, 0.6431, 0.6392],
...,
[0.7294, 0.7373, 0.7451, ..., 0.7333, 0.7294, 0.7255],
[0.7294, 0.7333, 0.7451, ..., 0.7373, 0.7333, 0.7255],
[0.7255, 0.7333, 0.7451, ..., 0.7373, 0.7294, 0.7216]]],

[[[0.5451, 0.5451, 0.5569, ..., 0.5765, 0.2745, 0.0000],
[0.5608, 0.5647, 0.5765, ..., 0.5882, 0.3412, 0.0000],
[0.5686, 0.5804, 0.5922, ..., 0.5529, 0.1882, 0.0000],
...,
[0.2471, 0.2392, 0.2471, ..., 0.5647, 0.6706, 0.7451],
[0.2353, 0.2314, 0.2275, ..., 0.7608, 0.7451, 0.7216],
[0.2235, 0.2275, 0.2275, ..., 0.6353, 0.7020, 0.7020]]],

...,

[[[0.5333, 0.5373, 0.5412, ..., 0.6471, 0.6471, 0.6471],
[0.5333, 0.5412, 0.5490, ..., 0.6471, 0.6471, 0.6471],
[0.5412, 0.5490, 0.5529, ..., 0.6510, 0.6510, 0.6510],
...,
[0.5961, 0.5961, 0.6039, ..., 0.7137, 0.7176, 0.7176],
[0.5961, 0.5961, 0.6078, ..., 0.7137, 0.7176, 0.7176],
[0.5922, 0.5961, 0.6078, ..., 0.7176, 0.7216, 0.7216]]],

[[[0.5412, 0.5451, 0.5529, ..., 0.5882, 0.5843, 0.5804],
[0.5451, 0.5529, 0.5647, ..., 0.5922, 0.5922, 0.5922],
[0.5490, 0.5608, 0.5647, ..., 0.5961, 0.5961, 0.5961],
...,
[0.3020, 0.3020, 0.3059, ..., 0.5529, 0.5765, 0.5294],
[0.3412, 0.3412, 0.3294, ..., 0.5529, 0.6000, 0.4275],
[0.3255, 0.3294, 0.3294, ..., 0.5725, 0.5608, 0.3765]]],

[[[0.5765, 0.5765, 0.5843, ..., 0.6706, 0.6706, 0.6667],
[0.5765, 0.5804, 0.5882, ..., 0.6745, 0.6667, 0.6667],
[0.5843, 0.5922, 0.5961, ..., 0.6235, 0.6784, 0.6706],
...,
[0.6471, 0.6314, 0.6235, ..., 0.7529, 0.7529, 0.7490],
[0.6549, 0.6118, 0.6392, ..., 0.7608, 0.7569, 0.7490],
[0.6471, 0.5922, 0.6863, ..., 0.7608, 0.7569, 0.7529]]],
device='cuda:0'),
tensor([16, 8, 20, 3, 0, 11, 8, 14, 11, 20, 17, 8, 9, 23, 1, 12, 22, 1,
3, 17, 10, 12, 10, 22, 20, 23, 7, 16, 17, 12, 18, 6],
device='cuda:0')]
```

It looks different, but let's check the `shape`s to be sure.

```
[11]: batch[0].shape
```

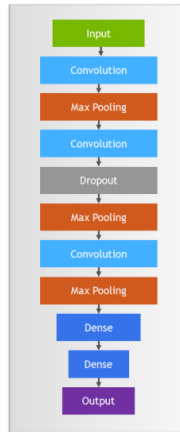
```
[11]: torch.Size([32, 1, 28, 28])
```

```
[12]: batch[1].shape
```

```
[12]: torch.Size([32])
```

3.3 Creating a Convolutional Model

These days, many data scientists start their projects by borrowing model properties from a similar project. Assuming the problem is not totally unique, there's a great chance that people have created models that will perform well which are posted in online repositories like [TensorFlow Hub](#) and the [NGC Catalog](#). Today, we'll provide a model that will work well for this problem.



We covered many of the different kinds of layers in the lecture, and we will go over them all here with links to their documentation. When in doubt, read the official documentation (or ask [Stack Overflow](#)).

```
[13]: n_classes = 24
kernel_size = 3
flattened_img_size = 75 * 3 * 3

model = nn.Sequential(
    # First convolution
    nn.Conv2d(IMG_CHS, 25, kernel_size, stride=1, padding=1), # 25 x 28 x 28
    nn.BatchNorm2d(25),
    nn.ReLU(),
    nn.MaxPool2d(2, stride=2), # 25 x 14 x 14
    # Second convolution
    nn.Conv2d(25, 50, kernel_size, stride=1, padding=1), # 50 x 14 x 14
    nn.BatchNorm2d(50),
    nn.ReLU(),
    nn.Dropout(.2),
    nn.MaxPool2d(2, stride=2), # 50 x 7 x 7
    # Third convolution
    nn.Conv2d(50, 75, kernel_size, stride=1, padding=1), # 75 x 7 x 7
    nn.BatchNorm2d(75),
    nn.ReLU(),
    nn.MaxPool2d(2, stride=2), # 75 x 3 x 3
    # Flatten to Dense
    nn.Flatten(),
    nn.Linear(flattened_img_size, 512),
    nn.Dropout(.3),
    nn.ReLU(),
    nn.Linear(512, n_classes)
)
```

3.3.1 Conv2D



These are our 2D convolutional layers. Small kernels will go over the input image and detect features that are important for classification. Earlier convolutions in the model will detect simple features such as lines. Later convolutions will detect more complex features. Let's look at our first Conv2D layer:

```
nn.Conv2d(IMG_CHS, 25, kernel_size, stride=1, padding=1)
```

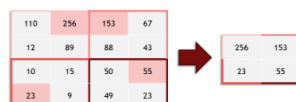
25 refers to the number of filters that will be learned. Even though `kernel_size = 3`, PyTorch will assume we want 3 x 3 filters. Stride refers to the step size that the filter will take as it passes over the image. Padding refers to whether the output image that's created from the filter will match the size of the input image.

3.3.2 BatchNormalization

Like normalizing our inputs, batch normalization scales the values in the hidden layers to improve training. [Read more about it in detail here.](#)

There is a debate on best where to put the batch normalization layer. [This Stack Overflow post](#) compiles many perspectives.

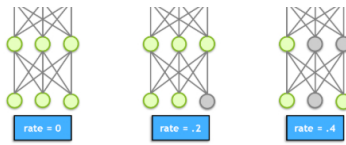
3.3.3 MaxPool2D



Max pooling takes an image and essentially shrinks it to a lower resolution. It does this to help the model be robust to translation (objects moving side to side), and also makes our model faster.

3.3.4 Dropout





Dropout is a technique for preventing overfitting. Dropout randomly selects a subset of neurons and turns them off, so that they do not participate in forward or backward propagation in that particular pass. This helps to make sure that the network is robust and redundant, and does not rely on any one area to come up with answers.

3.3.5 Flatten

Flatten takes the output of one layer which is multidimensional, and flattens it into a one-dimensional array. The output is called a feature vector and will be connected to the final classification layer.

3.3.6 Linear

We have seen dense linear layers before in our earlier models. Our first dense layer (512 units) takes the feature vector as input and learns which features will contribute to a particular classification. The second dense layer (24 units) is the final classification layer that outputs our prediction.

3.4 Summarizing the Model

This may feel like a lot of information, but don't worry. It's not critical that to understand everything right now in order to effectively train convolutional models. Most importantly we know that they can help with extracting useful information from images, and can be used in classification tasks.

```
[14]: model = torch.compile(model.to(device))
      model

[14]: OptimizedModule(
  (_orig_mod): Sequential(
    (0): Conv2d(1, 25, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): BatchNorm2d(25, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ReLU()
    (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (4): Conv2d(25, 50, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (5): BatchNorm2d(50, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (6): ReLU()
    (7): Dropout(p=0.2, inplace=False)
    (8): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (9): Conv2d(50, 75, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (10): BatchNorm2d(75, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (11): ReLU()
    (12): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (13): Flatten(start_dim=1, end_dim=-1)
    (14): Linear(in_features=675, out_features=512, bias=True)
    (15): Dropout(p=0.3, inplace=False)
    (16): ReLU()
    (17): Linear(in_features=512, out_features=24, bias=True)
  )
)
```

Since the problem we are trying to solve is still the same (classifying ASL images), we will continue to use the same `loss_function` and `accuracy` metric.

```
[15]: loss_function = nn.CrossEntropyLoss()
      optimizer = Adam(model.parameters())

[16]: def get_batch_accuracy(output, y, N):
      pred = output.argmax(dim=1, keepdim=True)
      correct = pred.eq(y.view_as(pred)).sum().item()
      return correct / N
```

3.5 Training the Model

Despite the very different model architecture, the training looks exactly the same.

Exercise

These are the same `train` and `validate` functions as before, but they have been mixed up. Can you correctly name each function and replace the `FIXME`s?

One of them should have `model.train` and the other should have `model.eval`.

```
[17]: def FIXME():
      loss = 0
      accuracy = 0

      model.FIXME()
      with torch.no_grad():
          for x, y in FIXME:
              output = model(x)

              loss += loss_function(output, y).item()
              accuracy += get_batch_accuracy(output, y, valid_N)
      print('FIXME - Loss: {:.4f} Accuracy: {:.4f}'.format(loss, accuracy))

[18]: def FIXME():
      loss = 0
      accuracy = 0

      model.FIXME()
      for x, y in FIXME:
          output = model(x)
          optimizer.zero_grad()
          batch_loss = loss_function(output, y)
          batch_loss.backward()
          optimizer.step()

          loss += batch_loss.item()
          accuracy += get_batch_accuracy(output, y, train_N)
      print('FIXME - Loss: {:.4f} Accuracy: {:.4f}'.format(loss, accuracy))
```

Solution

Click the two `...`s below for the solution.

```
# SOLUTION ***

# SOLUTION ***

[21]: epochs = 20

for epoch in range(epochs):
    print('Epoch: {}'.format(epoch))
    train()
    validate()
```

```
Epoch: 0
Train - Loss: 280.3605 Accuracy: 0.9020
Valid - Loss: 37.8465 Accuracy: 0.9410
Epoch: 1
Train - Loss: 10.8928 Accuracy: 0.9970
Valid - Loss: 15.0444 Accuracy: 0.9762
Epoch: 2
Train - Loss: 14.6257 Accuracy: 0.9947
Valid - Loss: 27.8649 Accuracy: 0.9621
Epoch: 3
Train - Loss: 15.6875 Accuracy: 0.9952
Valid - Loss: 38.7014 Accuracy: 0.9463
Epoch: 4
Train - Loss: 4.0912 Accuracy: 0.9986
Valid - Loss: 18.2867 Accuracy: 0.9774
Epoch: 5
Train - Loss: 8.5092 Accuracy: 0.9973
Valid - Loss: 9.3784 Accuracy: 0.9817
Epoch: 6
Train - Loss: 10.2783 Accuracy: 0.9965
Valid - Loss: 12.6313 Accuracy: 0.9817
Epoch: 7
Train - Loss: 3.5401 Accuracy: 0.9989
Valid - Loss: 29.2368 Accuracy: 0.9520
Epoch: 8
Train - Loss: 3.5931 Accuracy: 0.9988
Valid - Loss: 16.5711 Accuracy: 0.9823
Epoch: 9
Train - Loss: 9.0843 Accuracy: 0.9969
Valid - Loss: 17.8683 Accuracy: 0.9732
Epoch: 10
Train - Loss: 4.3765 Accuracy: 0.9987
Valid - Loss: 12.3995 Accuracy: 0.9753
Epoch: 11
Train - Loss: 0.1475 Accuracy: 1.0000
Valid - Loss: 10.4663 Accuracy: 0.9822
Epoch: 12
Train - Loss: 9.5791 Accuracy: 0.9974
Valid - Loss: 24.7274 Accuracy: 0.9668
Epoch: 13
Train - Loss: 2.3421 Accuracy: 0.9993
Valid - Loss: 21.4493 Accuracy: 0.9752
Epoch: 14
Train - Loss: 4.8510 Accuracy: 0.9985
Valid - Loss: 24.7301 Accuracy: 0.9736
Epoch: 15
Train - Loss: 4.8204 Accuracy: 0.9979
Valid - Loss: 31.0632 Accuracy: 0.9681
Epoch: 16
Train - Loss: 2.2360 Accuracy: 0.9992
Valid - Loss: 28.2613 Accuracy: 0.9707
Epoch: 17
Train - Loss: 4.9860 Accuracy: 0.9986
Valid - Loss: 17.0258 Accuracy: 0.9815
Epoch: 18
Train - Loss: 0.3036 Accuracy: 0.9999
Valid - Loss: 19.4492 Accuracy: 0.9787
Epoch: 19
Train - Loss: 3.4001 Accuracy: 0.9988
Valid - Loss: 37.9226 Accuracy: 0.9658
```

3.5.1 Discussion of Results

It looks like this model is significantly improved! The training accuracy is very high, and the validation accuracy has improved as well. This is a great result, as all we had to do was swap in a new model.

You may have noticed the validation accuracy jumping around. This is an indication that our model is still not generalizing perfectly. Fortunately, there's more that we can do. Let's talk about it in the next lecture.

3.6 Summary

In this section, we utilized several new kinds of layers to implement a CNN, which performed better than the more simple model used in the last section. Hopefully the overall process of creating and training a model with prepared data is starting to become even more familiar.

3.6.1 Clear the Memory

Before moving on, please execute the following cell to clear up the GPU memory. This is required to move on to the next notebook.

```
[22]: import IPython
app = IPython.Application.instance()
app.kernel.do_shutdown(True)
```

```
[22]: {'status': 'ok', 'restart': True}
```

3.6.2 Next

In the last several sections you have focused on the creation and training of models. In order to further improve performance, you will now turn your attention to *data augmentation*, a collection of techniques that will allow your models to train on more and better data than what you might have originally at your disposal.



[]:

